

Fundamentals of Software Engineering

Chapter 1 – Computer Fundamentals	3
What Is a Computer?	3
Characteristics of Computers	3
Types of Computers	4
Chapter 2 – Software Types	5
Software Defined	5
Categories of Software	5
Chapter 3 – Software Development Life Cycle (SDLC)	7
Why Is SDLC Necessary?	7
The 7 Stages of SDLC	7
Chapter 4 – Data Representation	9
Number Systems	9
Representing Negative Numbers: Two's Complement	9
Character Encoding	10
Chapter 5 – Boolean Logic	11
Boolean Algebra and Logic Gates	11
Chapter 6 – Security Fundamentals	12
The CIA Triad	12
Common Security Threats	12
Chapter 7 – Software Engineering Principles	14
What Is Software Engineering?	14
Core Software Engineering Principles	14
Development Methodologies	15
Agile Methodologies	16

Chapter 1 – Computer Fundamentals

What Is a Computer?

A **computer** is an electronic device that accepts data as input, processes it according to a stored set of instructions (a program), and produces output. What distinguishes a computer from a simple calculator is that its behavior can be changed by changing the program stored in its memory — it is **programmable**.

The concept of a "stored program computer" (where both program instructions and data reside in the same memory) was formalized by John von Neumann in 1945 — this **Von Neumann architecture** remains the foundation of virtually all modern computers:

- **CPU (Central Processing Unit)** — the brain; fetches, decodes, and executes instructions.
- **Memory (RAM)** — stores both the program being executed and the data it works on.
- **I/O subsystem** — allows interaction with the external world (keyboard, screen, disk).
- **System bus** — connects all components, carrying data, addresses, and control signals.

Characteristics of Computers

Computers excel at tasks that play to their innate strengths:

- **Speed** — modern CPUs execute billions of instructions per second (gigahertz clock speeds).
- **Accuracy** — computations are performed with perfect precision at the hardware level. Errors come from bad data or bad programs, not the hardware itself.
- **Storage capacity** — vast amounts of data can be stored on modern storage devices (terabytes on a single disk).
- **Automation** — once programmed, a computer can execute tasks repeatedly without human intervention.
- **Diligence** — computers do not tire, become bored, or lose concentration; they perform the same task with the same quality whether it's the first or the billionth iteration.

- **Versatility** — the same hardware can run a word processor, a game, a scientific simulation — any software.

Types of Computers

By size and power:

- **Supercomputers** — the most powerful computers in the world. Used for weather forecasting, molecular simulation, nuclear weapons modeling, and other massively parallel computations. Examples: IBM Summit, Fugaku.
 - **Mainframe computers** — large, powerful computers designed for processing enormous volumes of transactions. Used by banks, insurance companies, governments. Extremely reliable (designed for continuous operation). Less powerful than supercomputers but better at handling many simultaneous users.
 - **Minicomputers (Midrange computers)** — between mainframes and microcomputers. Used as department servers, engineering workstations. Now largely replaced by powerful servers.
 - **Microcomputers (Personal computers)** — desktop PCs, laptops — the computers most people use daily. Based on microprocessor chips.
 - **Embedded computers** — specialized computers built into devices for dedicated purposes. The "computer" inside a car's ABS braking system, a microwave, a washing machine. Not general-purpose.
-

Chapter 2 – Software Types

Software Defined

Software is the collection of programs and related data that tells a computer what to do. Unlike hardware (physical components), software is intangible – it is intellectual work stored as binary instructions. Without software, computer hardware is a useless collection of electronic components.

Categories of Software

System Software: Programs that manage and control the hardware, providing a platform for other software to run on.

- **Operating System (OS)** – the most important system software. It manages hardware resources, provides a user interface, and creates an environment for other programs to run. Examples: Windows, Linux, macOS, Android, iOS.
- **Device Drivers** – small programs that allow the OS to communicate with specific hardware devices (printer driver, graphics driver). Each hardware device needs a driver.
- **Utility Programs** – tools that help manage and maintain the computer. Examples: antivirus software, disk defragmenters, backup utilities, compression tools.

Application Software: Programs designed for end users to accomplish specific tasks.

Application software runs on top of system software.

- **General-purpose:** Microsoft Word (word processing), Excel (spreadsheets), Firefox (web browser).
- **Domain-specific:** Accounting software (QuickBooks), CAD software (AutoCAD), hospital management systems.

Programming Language Software: Tools that help programmers write code:

- **Compilers** — translate entire source code from a high-level language (C, Java) to machine code before execution.
 - **Interpreters** — translate and execute source code line-by-line at runtime (Python, JavaScript).
 - **IDEs (Integrated Development Environments)** — combine a code editor, compiler/interpreter, debugger, and other tools into a unified environment (VS Code, IntelliJ IDEA).
-

Chapter 3 – Software Development Life Cycle (SDLC)

Why Is SDLC Necessary?

Software development is a complex engineering process. Without a structured approach, projects become chaotic – budgets overrun, deadlines are missed, quality suffers, and software fails to meet user needs. The **SDLC** provides a systematic framework that guides the development process from initial idea to deployment and maintenance, ensuring quality and manageable progress.

The 7 Stages of SDLC

Stage 1 – Planning: The foundational stage. Define the problem to be solved, establish project scope, assess feasibility (technical, economic, legal, operational feasibility – **TELOS**), estimate costs and schedule, identify risks, and allocate resources. Without adequate planning, everything that follows is built on sand.

Stage 2 – Requirements Analysis: Gather detailed requirements by consulting with stakeholders (clients, users, domain experts). Distinguish between:

- **Functional requirements** – what the system must do (specific behaviors and functions).
- **Non-functional requirements** – how well the system must do it (performance, security, usability, reliability).

Output: a **Software Requirements Specification (SRS)** document – the contract between the development team and the client.

Stage 3 – System Design: Translate requirements into a blueprint for the system. This includes:

- **System architecture** – the high-level structure and components.
- **Database design** – data models, ER diagrams, normalization.
- **Interface design** – user interface mockups and API designs.

- **Algorithm design** – logic for complex computations.

Output: Design documents that guide the next stage.

Stage 4 – Implementation (Coding): Developers write the actual code according to the design specifications. Programming standards and guidelines are followed to ensure consistency and maintainability. This is often the most visible stage but is informed by all previous stages.

Stage 5 – Testing: Systematically verify that the software works correctly and meets requirements. Testing types: unit testing (individual components), integration testing (components together), system testing (entire system), acceptance testing (with real users). Bugs are identified, reported, fixed, and retested.

Stage 6 – Deployment: Release the tested software to the production environment. May involve installation, configuration, data migration, and user training. Deployment strategies vary: immediate cutover (risky), parallel running (old and new systems run side-by-side until confidence is established), phased rollout.

Stage 7 – Maintenance: The longest phase in the lifecycle – most software is maintained for many years after deployment. Four types of maintenance:

- **Corrective** – fix bugs discovered in production.
 - **Adaptive** – adapt to changes in the environment (new OS, new regulations).
 - **Perfective** – enhance features based on user feedback.
 - **Preventive** – refactor and improve internal structure to prevent future problems.
-

Chapter 4 – Data Representation

Number Systems

Computers store and process all information as binary (base-2) digits (bits). Understanding number systems is fundamental to understanding how computers work at the hardware level.

System	Base	Digits Used	Example
Binary	2	0, 1	1011_2
Octal	8	0-7	13_8
Decimal	10	0-9	11_{10}
Hexadecimal	16	0-9, A-F	B_{16}

Binary to Decimal: Multiply each bit by 2 raised to the power of its position (from right, starting at 0), then sum. $1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$

Decimal to Binary: Repeatedly divide by 2 and record remainders; read remainders bottom-to-top. $11 \div 2 = 5 \text{ R } 1 \rightarrow 5 \div 2 = 2 \text{ R } 1 \rightarrow 2 \div 2 = 1 \text{ R } 0 \rightarrow 1 \div 2 = 0 \text{ R } 1 \rightarrow \text{Result: } 1011_2$

Hexadecimal is used as shorthand for binary because every hex digit corresponds to exactly 4 bits: $B_{16} = 1011_2$ (B = 11 in decimal = 1011 in binary)

Representing Negative Numbers: Two's Complement

Computers represent negative integers using **two's complement** — the dominant method because it allows standard binary addition circuits to handle subtraction.

How to find two's complement of a number:

1. Write the positive version in binary.
2. **Invert all bits** (flip 0→1 and 1→0) — this gives the **one's complement**.
3. **Add 1** to the one's complement — this gives the two's complement.

Example: represent -5 in 8 bits:

- $+5 = 00000101$
- One's complement: 11111010
- Add 1: $11111011 = -5$ in two's complement

Properties:

- In an n -bit two's complement system, the range is $-2^{(n-1)}$ to $+2^{(n-1)}-1$.
- 8-bit range: -128 to $+127$.
- The most significant bit (MSB) is the sign bit: 1 means negative, 0 means non-negative.
- Subtraction is done as addition: $A - B = A + (\text{two's complement of } B)$.

Character Encoding

Text and symbols must also be represented as binary numbers. Several standards exist:

- **ASCII (American Standard Code for Information Interchange):** 7-bit code representing 128 characters (0–127). Covers English letters, digits, punctuation, and control characters. Extended ASCII uses 8 bits for 256 characters.
 - **EBCDIC (Extended Binary Coded Decimal Interchange Code):** An 8-bit IBM character encoding used on mainframe computers. Historically important but largely supplanted by ASCII and Unicode.
 - **Unicode:** A universal character encoding standard that represents virtually all characters from all the world's writing systems — over 143,000 characters. **UTF-8** is the most common Unicode encoding: variable-width (1–4 bytes per character), backward-compatible with ASCII for the first 128 characters.
-

Chapter 5 — Boolean Logic

Boolean Algebra and Logic Gates

Boolean algebra is the mathematics of logic, where values are either **true (1)** or **false (0)**. Computer hardware is built from **logic gates** – electronic circuits that implement Boolean operations. By combining millions of gates, CPUs perform all computations.

Basic Boolean Operations:

Operation	Symbol	Rule
AND	$A \cdot B$	Output is 1 only when BOTH inputs are 1
OR	$A + B$	Output is 1 when AT LEAST ONE input is 1
NOT	A' or $\neg A$	Output is the opposite of the input
XOR	$A \oplus B$	Output is 1 when inputs are DIFFERENT
NAND	$(A \cdot B)'$	AND followed by NOT – universal gate
NOR	$(A + B)'$	OR followed by NOT – universal gate

De Morgan's Laws (very important for exam questions):

- $(A \cdot B)' = A' + B'$ – NOT of AND = OR of NOTs
- $(A + B)' = A' \cdot B'$ – NOT of OR = AND of NOTs

In plain English: to negate a conjunction, negate each term and change AND to OR (and vice versa).

Truth tables are the canonical way to represent a Boolean function – list all possible input combinations (2^n rows for n inputs) and the corresponding output for each.

Chapter 6 – Security Fundamentals

The CIA Triad

The three core properties that every security system must protect are collectively called the **CIA Triad**:

- **Confidentiality** — information is accessible only to those authorized to access it. Ensured by: encryption, access control, authentication.
- **Integrity** — information is accurate and has not been tampered with by unauthorized parties. Ensured by: checksums, digital signatures, version control, audit logs.
- **Availability** — systems and information are accessible when needed by authorized users. Ensured by: redundancy, backups, failover systems, DDoS protection.

An attack that compromises any one of these properties is a security breach.

Common Security Threats

- **Virus** — malicious code that **attaches itself to a legitimate program** and spreads when that program is executed. It requires a host file to propagate.
- **Worm** — malicious code that **replicates itself** across networks without needing to attach to a host file. Worms spread automatically by exploiting network vulnerabilities.
- **Trojan Horse** — malicious code disguised as a legitimate, useful program. The user installs it willingly; it then performs malicious activities in the background (stealing data, creating backdoors).
- **Phishing** — a social engineering attack where attackers send fraudulent emails or messages pretending to be trusted entities (your bank, your employer) to trick you into revealing credentials, clicking malicious links, or downloading malware.
- **Rootkit** — software that hides the presence of other malware on a system, and often provides the attacker with privileged (root/admin) access that persists even through reboots.
- **Keylogger** — software or hardware that records all keystrokes, capturing passwords, credit card numbers, and other sensitive information.
- **Ransomware** — encrypts the victim's files and demands payment (ransom) for the decryption key. Typically spread via phishing emails or compromised websites.

Countermeasures: Antivirus software, firewalls, regular patches and updates, strong passwords with multi-factor authentication (MFA), user education, data backups, network monitoring.

Chapter 7 – Software Engineering Principles

What Is Software Engineering?

Software Engineering is the systematic application of engineering principles to the design, development, testing, deployment, and maintenance of software systems. It treats software

development as an engineering discipline — not just programming — emphasizing process, methodology, quality, and professionalism.

The term was deliberately chosen to reflect that software development should be subject to the same rigor as civil or electrical engineering. Just as bridges are designed with safety margins and proven techniques, software should be built with proven methodologies that ensure correctness, reliability, and maintainability.

Core Software Engineering Principles

- **Modularity** — break large systems into smaller, independent modules. Each module has a well-defined responsibility and interface. Benefits: easier to understand, test, modify, and reuse individual pieces.
- **Abstraction** — hide unnecessary details; expose only what is needed. Good abstractions let users work at the right level of complexity.
- **Separation of Concerns** — different aspects of the system (UI, business logic, data storage) should be handled by separate components.
- **Information Hiding (Encapsulation)** — modules should hide their internal implementation. Other modules interact only through a defined interface. This limits the blast radius of changes.
- **Cohesion** — each module should have a single, focused purpose (high cohesion is good).
- **Coupling** — modules should be as independent as possible (low coupling is good). Highly coupled modules are hard to change without breaking others.

Development Methodologies

The methodology defines the overall process and philosophy of how development is organized.

Waterfall Model: A sequential, linear model where each phase must be completed before the next begins. Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance.

Strengths: Simple, easy to understand, well-documented, good for projects with well-understood and stable requirements. Weaknesses: Very inflexible — changes in

requirements late in the process are expensive; working software is not available until very late; testing happens after coding, so bugs are discovered late when they are expensive to fix. Not suitable for complex, innovative, or evolving requirements.

Iterative Model: Development proceeds in repeated cycles (iterations), with each iteration producing a working (though incomplete) version of the product. Requirements can be refined between iterations. Allows feedback and adaptation.

Prototyping Model: Build a quick, partial implementation (**prototype**) early to demonstrate key functionality and get user feedback before building the full system. Two types:

- **Throw-away prototype** — built quickly to clarify requirements, then discarded; the real system is built from scratch.
- **Evolutionary prototype** — the prototype is refined and extended until it becomes the final system.

Prototyping is powerful for UI design and novel systems where requirements are unclear.

Spiral Model: Combines iterative development with the structured risk analysis approach of the waterfall model. Each spiral loop consists of 4 quadrants:

1. Determine objectives and constraints.
2. Evaluate alternatives, identify and resolve risks.
3. Develop and verify the deliverable for this cycle.
4. Plan the next cycle.

Best for large, complex, high-risk projects. Risk management is a first-class concern at every step.

Agile Methodologies

Agile is a philosophy of software development that values:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.

- **Responding to change** over following a plan.

These values are stated in the **Agile Manifesto** (2001). Agile does not mean "no process" — it means lightweight, adaptive processes.

Scrum (a popular Agile framework): Work is divided into **Sprints** — fixed-length iterations (typically 1–4 weeks). Each sprint produces a potentially shippable product increment.

Key roles:

- **Product Owner** — defines and prioritizes the product backlog (the list of all features to build); represents the customer.
- **Scrum Master** — facilitates the team; removes impediments; ensures Scrum is followed correctly.
- **Development Team** — cross-functional team that does the actual work.

Key ceremonies:

- **Sprint Planning** — team selects items from the backlog to work on in the upcoming sprint.
- **Daily Scrum (Daily Standup)** — 15-minute daily meeting: what did I do yesterday? What will I do today? Any impediments?
- **Sprint Review** — demonstrate the working software to stakeholders.
- **Sprint Retrospective** — team reflects on the process: what went well, what to improve.

XP (Extreme Programming): Emphasizes technical excellence and short feedback cycles. Key practices: Test-Driven Development (TDD — write tests before code), pair programming (two developers work at one computer), continuous integration, small releases, refactoring, simple design. XP is best for small teams with rapidly changing requirements.

RAD (Rapid Application Development): Emphasizes rapid prototyping and iterative delivery with heavy user involvement. Relies on visual development tools and code reuse to accelerate development. Prioritizes speed of delivery over perfect architecture. Trades quality for speed — appropriate when time-to-market is critical.

